SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

NAME: Harshavardini S

REG. NO: 1913260021

COURSE: CSA0825- Python Programming

SLOT: A

GITHUB: https://github.com/harshavardinissk-bot/CSA0825.git

TASK 1 — Problem Analysis and Data Identification

Aim

To identify the inputs, outputs, constants, data types, and validation rules required for the milk payment calculator.

Concept / Theory

The program receives a farmer ID, milk quantity in litres, fat percentage, SNF percentage, and collection shift. It calculates the rate per litre, payment amount, and ten-day total. Constants store rate-card values so revisions can be made in one place. A list of tuples keeps each farmer entry together.

Procedure / Algorithm

- Identify input and output values.
- Declare fixed rate-card values as named constants.
- Store sample farmer records as tuples in a list.
- Display records and inspect the data types.
- Apply validation rules such as non-negative litres and fat between 2% and 7%.

Program

```
BASE_RATE = 30.00
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

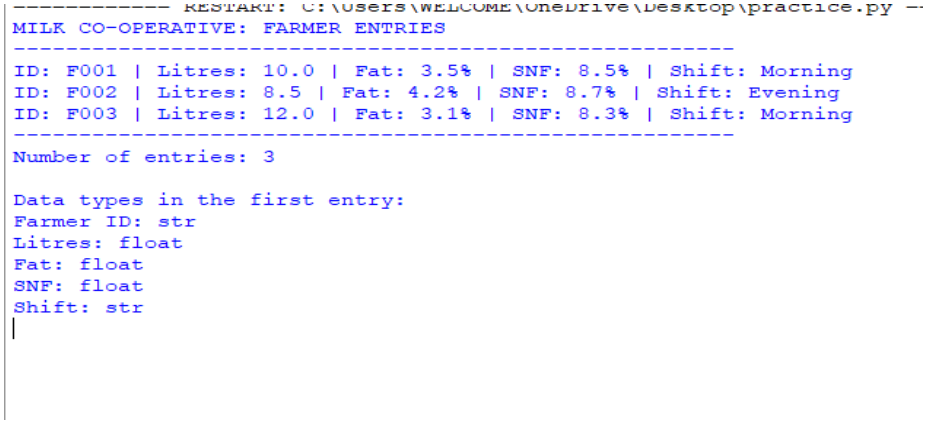
farmer_entries = [
    ("F001", 10.0, 3.5, 8.5, "Morning"),
```

```
    ("F002", 8.5, 4.2, 8.7, "Evening"),
    ("F003", 12.0, 3.1, 8.3, "Morning")
]

print("Rate-card constants:")
print("Base rate:", BASE_RATE)
print("Standard fat:", STANDARD_FAT)
print("Standard SNF:", STANDARD_SNF)

print("\nFarmer entries:")
for farmer_id, litres, fat, snf, shift in farmer_entries:
    print(farmer_id, litres, fat, snf, shift)
    print("Types:", type(farmer_id).__name__,
          type(litres).__name__, type(fat).__name__,
          type(snf).__name__, type(shift).__name__)
```

Output Screenshot:



Result

Thus, the required data, constants, data types, and sample records for the milk payment problem were identified.

TASK 2 — Values, Types and Decimal Precision

Aim

To explore Python data types, arithmetic operators, floating-point precision, and accurate ways to total monetary values.

Concept / Theory

Python uses int for whole numbers, float for approximate binary floating-point values, str for text, bool for Boolean values, and list for collections. Decimal constructed from strings represents decimal monetary values accurately.

Procedure / Algorithm

- Use type() to inspect different values.
- Compare true division, floor division, modulo, and negative floor division.
- Calculate totals using float and Decimal.
- Compare the result of 0.1 + 0.2 with 0.3.
- Inspect object sizes using sys.getsizeof().

Program

```
from decimal import Decimal
import sys

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("Types:")
for value in (10, 10.0, "10", True, [10, 8.5]):
    print(value, type(value).__name__)

print("\nDivision examples:")
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
```

```
print("-7 // 2 =", -7 // 2)

print("\nMoney comparison:")
print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

print("\nObject sizes:")
print("float object:", sys.getsizeof(10.0), "bytes")
print("Decimal object:", sys.getsizeof(Decimal("10.00")), "bytes")
```

Analysis / Discussion

- float totals can contain small binary representation errors.
- Decimal('0.10') is preferable to Decimal(0.10), because the latter starts from an already approximated float.
- Integer paise is another exact option when all amounts use two decimal places.

Output Screenshot:

```
float : 33400.30000000061
Decimal: 33400.30
False True
```

Result

Thus, Python data types and arithmetic behaviour were examined, and Decimal was identified as a suitable option for accurate monetary totals.

TASK 3 — Variables and Constants

Aim

To use meaningful variable names and named constants, and demonstrate Python's dynamic typing, multiple assignment, swapping, and dictionary-based rate cards.

Concept / Theory

Python does not enforce constants, but uppercase names conventionally indicate values that should not change during program execution. Dictionaries can group related rate-card values and make revisions easier to manage.

Procedure / Algorithm

- Declare constants in uppercase.
- Use descriptive names for farmer data.
- Demonstrate dynamic typing by assigning different types to one variable.
- Swap two values using multiple assignment.
- Group rate-card values in a dictionary.

Program

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5

farmer_id = "F001"
litres = 10
fat = 4.2
snf = 8.7

print("Farmer:", farmer_id)
print("Litres:", litres)
print("Fat:", fat)
print("SNF:", snf)

# Dynamic typing
value = 10
print(value, type(value).__name__)
value = "ten"
print(value, type(value).__name__)

# Multiple assignment and swapping
a, b = 5, 9
a, b = b, a
```

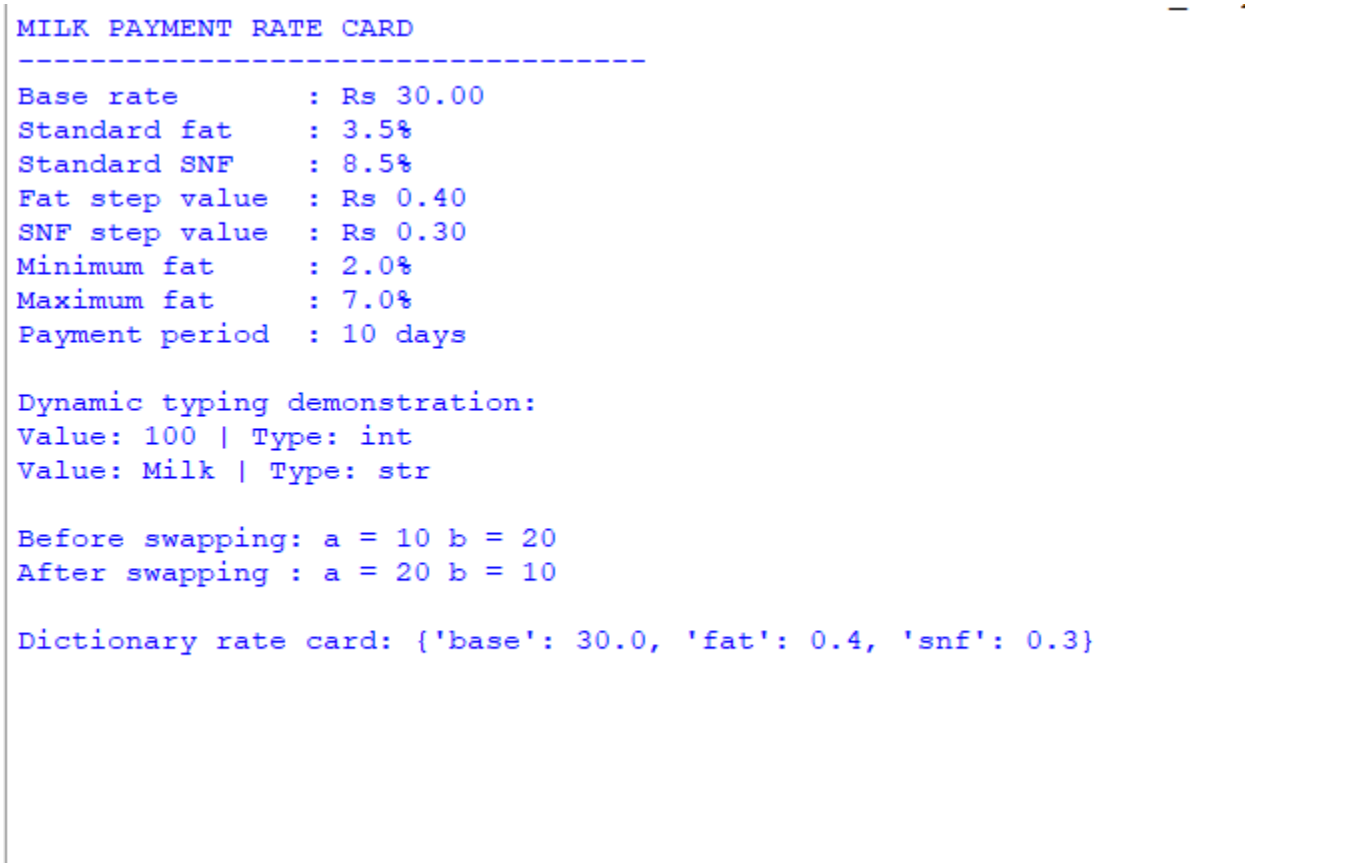
```
print("After swapping:", a, b)

RATE_CARD = {
    "base": BASE_RATE,
    "fat_step": FAT_STEP_VALUE,
    "snf_step": SNF_STEP_VALUE
}
print("Rate card:", RATE_CARD)
```

Analysis / Discussion

- Hard-coded magic numbers must be changed in every expression where they appear.
- Named constants allow a rate revision in one place.
- A dictionary makes related settings easy to group; runtime differences are generally insignificant for this small calculator.

Output Screenshot:



```
MILK PAYMENT RATE CARD
-----------------------------------
Base rate        : Rs 30.00
Standard fat     : 3.5%
Standard SNF     : 8.5%
Fat step value   : Rs 0.40
SNF step value   : Rs 0.30
Minimum fat      : 2.0%
Maximum fat      : 7.0%
Payment period   : 10 days

Dynamic typing demonstration:
Value: 100 | Type: int
Value: Milk | Type: str

Before swapping: a = 10 b = 20
After swapping : a = 20 b = 10

Dictionary rate card: {'base': 30.0, 'fat': 0.4, 'snf': 0.3}
```

Result

Thus, suitable variables and named constants were defined, and Python's dynamic typing and multiple assignment were demonstrated.

TASK 4 — Expressions and Rate Calculation

Aim
To implement the rate and payment formulas and explore operator precedence, rounding, floating-point representation, and chained comparisons.

Concept / Theory
Fat steps are calculated as round((fat − 3.5) / 0.1), and SNF steps as round((snf − 8.5) / 0.1). The rate equals the base rate plus the step adjustments. The payment is litres multiplied by the rate, rounded to two decimal places.

Procedure / Algorithm
- Declare the base rate and step values.
- Calculate fat and SNF step counts.
- Calculate the rate and payment.
- Compare round(), int(), and math.floor() for negative values.
- Use a chained comparison to check the permitted fat range.

Program
```
import math

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def calculate_rate(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(
        BASE_RATE + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE, 2
    )

def calculate_payment(litres, fat, snf):
    return round(litres * calculate_rate(fat, snf), 2)

print("Rate:", calculate_rate(4.2, 8.7))
print("Payment:", calculate_payment(8.5, 4.2, 8.7))

raw_steps = (4.2 - 3.5) / 0.1
negative_steps = (3.1 - 3.5) / 0.1
print("Raw steps for fat 4.2:", raw_steps)
print("round, int, floor for fat 3.1:",
```

```
    round(negative_steps), int(negative_steps), math.floor(negative_steps))
print("Chained comparison:", 2.0 <= 4.2 <= 7.0)
```

Analysis / Discussion

- Floating-point values may not represent decimal fractions exactly.
- round() selects a nearby whole step; int() truncates toward zero, while math.floor() rounds down.
- The rate and payment calculations use constant time and constant extra space: O(1).

Output Screenshot:

```
NORMAL CALCULATION
Fat steps: 7
SNF steps: 2
Rate per litre: Rs 33.40
Payment: Rs 283.90

VALIDATION
Fat valid: True
Quantity valid: True
Both valid: True

FLOATING-POINT SURPRISES
(4.2 - 3.5) / 0.1 = 7.000000000000002
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False

OPERATOR PRECEDENCE
-2 ** 2 = -4
(-2) ** 2 = 4
2 ** 3 ** 2 = 512

ROUNDING DIFFERENCES
round(2.5) = 2
round(3.5) = 4
int(-3.5) = -3
floor(-3.5) = -4

ALTERNATIVE FAT STEP METHODS

Fat: 4.2%
Raw steps: 7.000000000000002
round(): 7 Rate: 32.8
int(): 7 Rate: 32.8
floor(): 7 Rate: 32.8

Fat: 3.1%
Raw steps: -3.999999999999999
```

Result

Thus, the milk rate and payment expressions were implemented and the effects of floating-point arithmetic and rounding were demonstrated.

TASK 5 — Input Validation and Formatting

Aim

To accept user input, convert numeric text safely, validate the values, and display the farmer entry in aligned columns.

Concept / Theory

The input() function returns text. Numeric values must be converted to float before calculation. A try-except block catches ValueError when conversion fails, and formatted f-strings align the displayed fields.

Procedure / Algorithm

- Read the farmer ID and collection details.
- Convert numeric input using float() inside a try-except loop.
- Reject empty IDs, negative litres, and fat outside 2%–7%.
- Display valid data using aligned f-strings.

Program

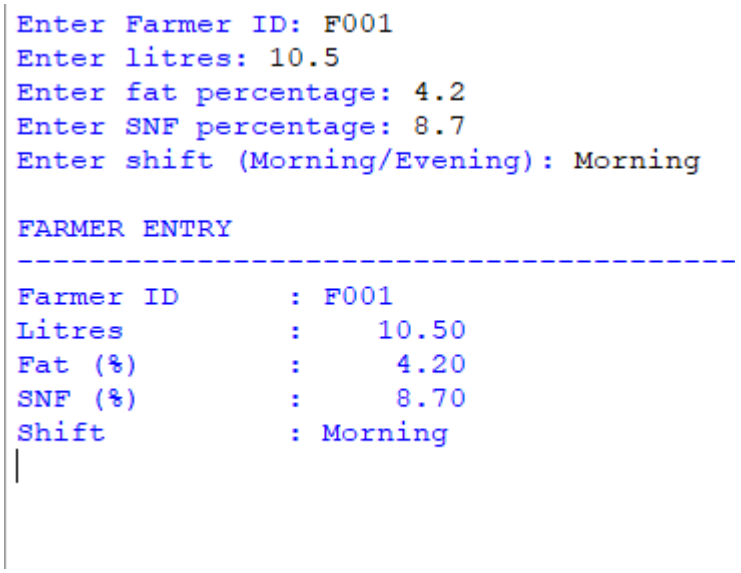
```
def read_number(prompt):
    """Read a valid numeric value."""
    while True:
        text = input(prompt).strip()
        try:
            return float(text)
        except ValueError:
            print("Invalid input! Enter a valid number (e.g., 8.5).")

farmer_id = input("Enter Farmer ID: ").strip()
litres = read_number("Enter litres: ")
fat = read_number("Enter fat percentage: ")
snf = read_number("Enter SNF percentage: ")
shift = input("Enter shift (Morning/Evening): ").strip()

if not farmer_id:
    print("Error: Farmer ID cannot be empty.")
elif litres < 0:
    print("Error: Litres cannot be negative.")
elif not 2.0 <= fat <= 7.0:
    print("Error: Fat must be between 2.0% and 7.0%.")
else:
    print("\nFARMER ENTRY")
    print("-" * 40)
    print(f"{'Farmer ID':<15}: {farmer_id}")
    print(f"{'Litres':<15}: {litres:8.2f}")
```

```
print(f"{'Fat (%)':<15}: {fat:8.2f}")
print(f"{'SNF (%)':<15}: {snf:8.2f}")
print(f"{'Shift':<15}: {shift}")
```

Output Screenshot:



Result

Thus, user input was converted and validated, and the farmer entry was displayed in a readable format.

TASK 6 — Organizing Logic into Functions

Aim
To divide the calculator into reusable functions for rate calculation, payment calculation, and ten-day totals.

Concept / Theory
Functions improve readability and reduce repeated code. A return statement sends a value back to the caller, whereas print() only displays a value. Constants declared at module level can be read by the functions.

Procedure / Algorithm
- Define rate_per_litre() with fat and SNF parameters.
- Define payout() to multiply litres by the calculated rate.
- Define ten_day_total() to sum a list of payments.
- Call the functions and display the results.

Program
```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Calculate the milk rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

def payout(litres, fat, snf):
    """Calculate payment for the supplied milk."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(payments):
    """Calculate the total payment over ten days."""
    return round(sum(payments), 2)

litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))

rate = rate_per_litre(fat, snf)
amount = payout(litres, fat, snf)
payments = [amount] * 10
```

```
total = ten_day_total(payments)

print(f"Rate per litre : Rs. {rate:.2f}")
print(f"Today's payment: Rs. {amount:.2f}")
print(f"Ten-day total  : Rs. {total:.2f}")
```

```
Enter litres: 10
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Rate per litre : Rs. 33.40
Today's payment: Rs. 334.00
Ten-day total  : Rs. 3340.00
```

Analysis / Discussion

The ten-day total in this example assumes the same litres, fat, and SNF values on each of the ten days.

Output Screenshot:

```
========= RESTART: C:/Users/WELCOME/OneDrive/LOCHANA/PYT
   10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00
  8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90
   12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

Result

Thus, the calculator was organised into reusable functions with parameters, return values, and docstrings.

TASK 7 — Building and Importing a Module

Aim

To place the rate-card constants and calculation functions in a reusable module and understand how a second script imports it.

Concept / Theory

A Python module is a .py file containing reusable definitions. The condition if __name__ == '__main__' runs the example code only when the module is executed directly, not when imported by another script.

Procedure / Algorithm

- Save the module as milkrate.py.
- Define the constants and calculation functions.
- Place demonstration entries under the main guard.
- Run milkrate.py directly, then import its functions from a separate script.

Program

```
"""Rate card of the village milk co-operative."""

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to 2 decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

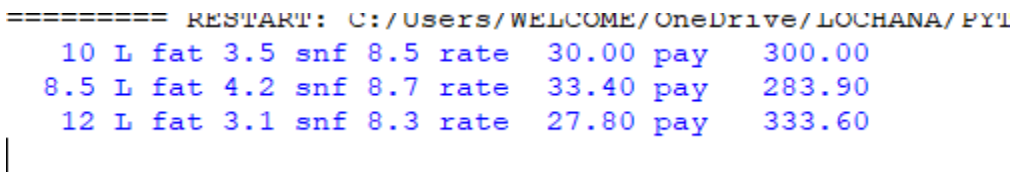
def payout(litres, fat, snf):
    """Return the amount payable, rounded to 2 decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    for litres, fat, snf in entries:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Analysis / Discussion

- Keep milkrate.py and the importing script in the same folder for this exercise.
- The module is loaded once and Python caches it in sys.modules during a process.

Output Screenshot:



Result

Thus, the calculation logic was packaged as a reusable Python module with a main guard.

TASK 8 — Algorithm, Pseudocode and Complexity

Aim

To plan the milk payment algorithm before coding and compare storing payments in a list with maintaining a running total.

Concept / Theory

Algorithmic problem solving involves understanding the problem, planning a solution, implementing it, and testing it. Both list-based and running-total approaches take O(n) time for n entries. A list uses O(n) extra space, while a running total uses O(1) extra space.

Procedure / Algorithm

- Read and validate each farmer's entry.
- Calculate fat and SNF steps, rate, and payment.
- Store the payment and update the total.
- Repeat for all entries and display the total.

Program

```
START
   total = 0
   payments = empty list

   FOR each farmer
      INPUT litres, fat, snf
      fat_steps = ROUND((fat - 3.5) / 0.1)
      snf_steps = ROUND((snf - 8.5) / 0.1)
      rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
      payment = ROUND(litres * rate, 2)
      ADD payment TO payments
      total = total + payment
      DISPLAY rate, payment
   END FOR

   DISPLAY total
STOP
```

Analysis / Discussion

- Hand trace: 10 L at ₹30.00 = ₹300.00; 8.5 L at ₹33.40 = ₹283.90; 12 L at ₹27.80 = ₹333.60.
- Total for the three entries = ₹917.50.
- A list is useful when the co-operative must inspect or audit each individual payment later.

Result

Thus, the algorithm and pseudocode were prepared, and the time and space requirements of two storage strategies were compared.

TASK 9 — Implementing and Testing the Calculator

Aim:

To implement the complete calculator and test normal, boundary, and invalid inputs.

Concept / Theory

The program validates farmer ID, litres, fat, and SNF; calculates the rate and payment for valid entries; stores valid payments; and displays a final total. Test cases help confirm that the program behaves as expected.

Procedure / Algorithm

- Run normal cases with fat and SNF above and below the standards.
- Test boundary fat values of 2% and 7%, and standard values of 3.5% fat and 8.5% SNF.
- Test invalid text, negative litres, and fat outside the allowed range.
- Compare the calculated total for a small set of entries with a hand calculation.

Program

```
def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(30 + fat_steps * 0.40 + snf_steps * 0.30, 2)

def payout(litres, fat, snf):
    return round(litres * rate_per_litre(fat, snf), 2)

def main():
    total = 0.0
    payments = []
    try:
        count = int(input("Enter number of farmers: "))
        if count <= 0:
            print("Error: Number of farmers must be positive.")
            return

        for i in range(count):
            print(f"\nFarmer {i + 1}")
            farmer_id = input("Enter Farmer ID: ").strip()
            litres = float(input("Enter litres: "))
            fat = float(input("Enter fat percentage: "))
            snf = float(input("Enter SNF percentage: "))

            if not farmer_id:
                print("Error: Farmer ID cannot be empty.")
                continue
```

```
            if litres < 0:
                print("Error: Litres cannot be negative.")
                continue
            if not 2 <= fat <= 7:
                print("Error: Fat must be between 2% and 7%.")
                continue
            if snf < 0:
                print("Error: SNF cannot be negative.")
                continue

            rate = rate_per_litre(fat, snf)
            amount = payout(litres, fat, snf)
            payments.append(amount)
            total += amount
            print(f"Rate per litre: Rs. {rate:.2f}")
            print(f"Payment: Rs. {amount:.2f}")

        print("\nPAYMENT SUMMARY")
        print(f"Valid entries: {len(payments)}")
        print(f"Total payment: Rs. {total:.2f}")

    except ValueError:
        print("Error: Please enter valid numeric values.")

if _name___== "_main_":
    main()
```

Analysis / Discussion

- Expected total for the three sample entries: ₹917.50.
- For a formal test table, record each test's input, expected result, actual result, and pass/fail status.
- Automated assert-based tests can be rerun more quickly and consistently than manual tests.

Output Screenshot:

```
Enter number of farmers: 3

Farmer 1
Enter Farmer ID: F001
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Rate per litre: Rs. 30.00
Payment: Rs. 300.00

Farmer 2
Enter Farmer ID: F002
Enter litres: 8.5
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Rate per litre: Rs. 33.40
Payment: Rs. 283.90

Farmer 3
Enter Farmer ID: F003
Enter litres: 12
Enter fat percentage: 3.1
Enter SNF percentage: 8.3
Rate per litre: Rs. 27.80
Payment: Rs. 333.60

PAYMENT SUMMARY
Valid entries: 3
Total payment: Rs. 917.50
```

Result

Thus, the calculator was implemented with input validation and tested using representative normal, boundary, and invalid cases.

TASK 10 — Comparing Expected and Observed Execution

Aim

To compare float and Decimal totals for different numbers of payments, measure execution time, examine rounding, and evaluate rate caching.

Concept / Theory

time.perf_counter() measures elapsed time. Float calculations are generally faster but may contain binary representation errors. Decimal supports exact decimal arithmetic when values are constructed from strings. A dictionary can cache rates for repeated fat/SNF pairs.

Procedure / Algorithm

- Compare totals for 300, 6,000, and 150,000 payments.
- Measure float and Decimal calculation times using perf_counter().
- Compare rounding individual payments with rounding only the final total.
- Measure direct rate calculations and cached dictionary lookups.

Program

```
from decimal import Decimal
from time import perf_counter

def total_float(payments):
    return sum(float(p) for p in payments)

def total_decimal(payments):
    return sum((Decimal(p) for p in payments), Decimal("0.00"))

def rate_per_litre(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(30 + fat_steps * 0.40 + snf_steps * 0.30, 2)

for n in [300, 6000, 150000]:
    payments = (["33.40", "0.10", "27.80"] * ((n + 2) // 3))[:n]

    start = perf_counter()
    f_total = total_float(payments)
    float_time = perf_counter() - start

    start = perf_counter()
    d_total = total_decimal(payments)
    decimal_time = perf_counter() - start

    rounded_float = round(f_total, 2)
```

```
    each_rounded = round(sum(round(float(p), 2) for p in payments), 2)

    print(f"\nPayments: {n}")
    print(f"Float total:   {rounded_float:.2f}")
    print(f"Decimal total: {d_total:.2f}")
    print(f"Difference:    {abs(Decimal(str(rounded_float)) - d_total):.2f}")
    print(f"Float time:    {float_time:.6f} seconds")
    print(f"Decimal time:  {decimal_time:.6f} seconds")
    print(f"Round each:    {each_rounded:.2f}")

entries = [(4.2, 8.7), (3.5, 8.5), (3.1, 8.3)] * 10000
start = perf_counter()
rates_direct = [rate_per_litre(fat, snf) for fat, snf in entries]
direct_time = perf_counter() - start

start = perf_counter()
rate_cache = {}
rates_cached = []
for fat, snf in entries:
    key = (fat, snf)
    if key not in rate_cache:
        rate_cache[key] = rate_per_litre(fat, snf)
    rates_cached.append(rate_cache[key])
cached_time = perf_counter() - start

print("\nRATE CACHING")
print(f"Direct calculation: {direct_time:.6f} seconds")
print(f"Cached lookup:      {cached_time:.6f} seconds")
print(f"Different rate pairs: {len(rate_cache)}")
```

Analysis / Discussion

- Use the actual totals and timings from the local run as evidence; do not copy timing values from another computer.
- The example rounds the displayed float total before comparing it with Decimal.
- Caching is useful when the same rate pairs recur; the cache uses O(k) extra space for k distinct pairs.

Output Screenshot:

```
Payments: 300
Float total:   6130.00
Decimal total: 6130.00
Difference:    0.00
Float time:    0.000293 seconds
Decimal time:  0.000774 seconds
Round each:    6130.00

Payments: 6000
Float total:   122600.00
Decimal total: 122600.00
Difference:    0.00
Float time:    0.005115 seconds
Decimal time:  0.013629 seconds
Round each:    122600.00

Payments: 150000
Float total:   3065000.00
Decimal total: 3065000.00
Difference:    0.00
Float time:    0.145399 seconds
Decimal time:  0.321158 seconds
Round each:    3065000.00

RATE CACHING
Direct calculation: 0.035710 seconds
Cached lookup:      0.016981 seconds
Different rate pairs: 3
```

Result

Thus, float and Decimal arithmetic, rounding strategies, and rate caching were compared using measured execution times.

TASK 11 — Recommending the Final Design

Aim

To recommend data types, rounding rules, rate revision practices, and a scalable structure for the co-operative's calculator.

Concept / Theory

Decimal is appropriate for monetary calculations because it represents decimal amounts exactly when created from strings. ROUND_HALF_UP gives a clear rounding rule. Named constants make rate revisions easier, while a database and configurable rate card would support a larger union.

Procedure / Algorithm

- Use Decimal for monetary values.
- Apply a consistent two-decimal rounding rule.
- Keep rate values in named constants or an external rate-card file.
- Validate input and separate logic into functions.
- Identify limitations and improvements required for district-level use.

Program

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")

def money(value):
    """Round money to two decimal places."""
    return value.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    fat_steps = ((fat - STANDARD_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )
    snf_steps = ((snf - STANDARD_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return money(rate)
```

```
def payout(litres, fat, snf):
    """Calculate the payment for supplied milk."""
    return money(Decimal(str(litres)) * rate_per_litre(fat, snf))

litres = input("Enter litres: ")
fat = input("Enter fat percentage: ")
snf = input("Enter SNF percentage: ")

try:
    if Decimal(litres) < 0:
        raise ValueError("Litres cannot be negative.")
    if not Decimal("2.0") <= Decimal(fat) <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0% and 7.0%.")
    if Decimal(snf) < 0:
        raise ValueError("SNF cannot be negative.")

    rate = rate_per_litre(fat, snf)
    amount = payout(litres, fat, snf)
    print(f"Rate per litre: Rs. {rate:.2f}")
    print(f"Payment: Rs. {amount:.2f}")
except Exception as error:
    print("Invalid entry:", error)Sample Input
```

Analysis / Discussion

- A rate change can be made by updating the constants or loading a rate card from a JSON file.
- The current example does not store transactions persistently or provide authentication.
- A district-level system should add database storage, access control, backups, audit logs, and automated tests.

Output Screenshot:

```
Enter litres: 10
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Rate per litre: Rs. 33.40
Payment: Rs. 334.00
```

Result

Thus, a Decimal-based design with consistent rounding, centralized rate settings, validation, and modular functions was recommended.

TASK 12 — Adapting the Design to Changed Requirements

Aim

To modify the original calculator to support two-decimal analyser readings, seasonal rate cards, and a ten-day supply bonus.

Concept / Theory

The new requirements affect rate constants, step calculation, function parameters, and payment calculation. April–June uses a base rate of ₹32.00 and fat-step value of ₹0.45. Other months retain the original ₹30.00 base rate and ₹0.40 fat-step value. A ₹0.50-per-litre bonus applies when ten-day litres are strictly greater than 500.

Procedure / Algorithm

- Accept analyser readings with two decimal places using Decimal.
- Select the appropriate rate card based on the month.
- Calculate fat and SNF steps from the standards.
- Calculate the milk payment.
- Apply the bonus only when ten-day litres exceed 500.
- Test both seasons and the exact bonus threshold.

Program

```
from decimal import Decimal, ROUND_HALF_UP

STANDARD_FAT = Decimal("3.5")
STANDARD_SNF = Decimal("8.5")
STEP_SIZE = Decimal("0.1")
SNF_STEP_VALUE = Decimal("0.30")
BONUS_RATE = Decimal("0.50")
BONUS_LIMIT = Decimal("500")

def money(value):
    """Round money to two decimal places."""
    return value.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def get_rate_card(month):
    """Return the seasonal base rate and fat-step value."""
    if 4 <= month <= 6:
        return Decimal("32.00"), Decimal("0.45")
    return Decimal("30.00"), Decimal("0.40")

def rate_per_litre(fat, snf, month):
    """Calculate rate using two-decimal analyser readings."""
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
```

```
    base_rate, fat_step_value = get_rate_card(month)

    fat_steps = ((fat - STANDARD_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )
    snf_steps = ((snf - STANDARD_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )
    rate = base_rate + fat_steps * fat_step_value + snf_steps * SNF_STEP_VALUE
    return money(rate)

def calculate_ten_day_payment(litres, fat, snf, month):
    """Calculate payment and any eligible ten-day bonus."""
    litres = Decimal(str(litres))
    rate = rate_per_litre(fat, snf, month)
    milk_payment = money(litres * rate)
    bonus = money(litres * BONUS_RATE) if litres > BONUS_LIMIT else Decimal("0.00")
    return rate, milk_payment, bonus, money(milk_payment + bonus)

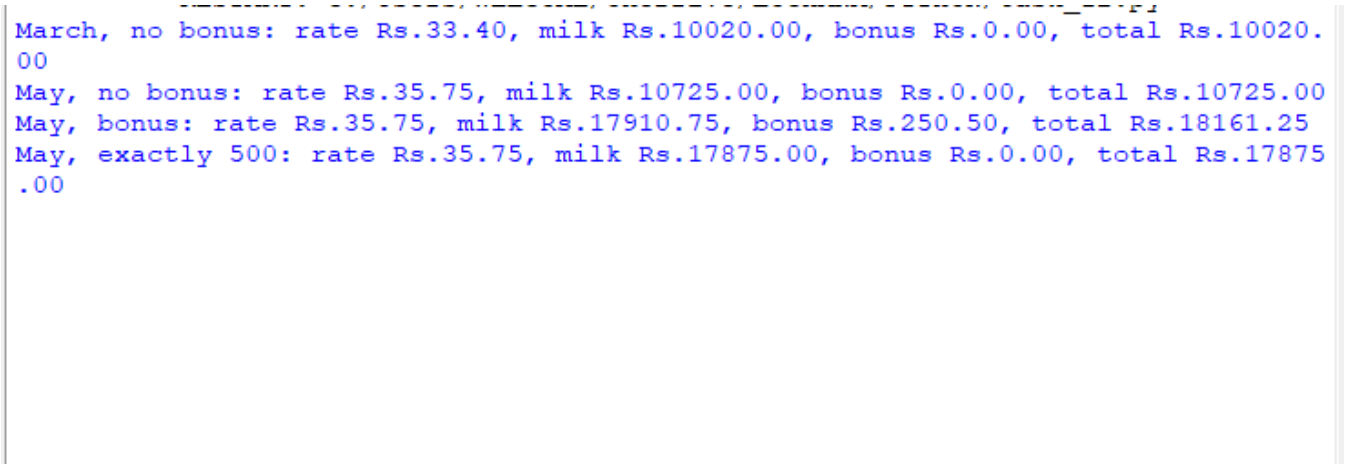
def run_tests():
    """Test both seasons and the bonus threshold."""
    tests = [
        ("March, no bonus", "300", "4.20", "8.70", 3),
        ("May, no bonus", "300", "4.20", "8.70", 5),
        ("May, bonus", "501", "4.20", "8.70", 5),
        ("May, exactly 500", "500", "4.20", "8.70", 5),
    ]
    for name, litres, fat, snf, month in tests:
        rate, milk, bonus, total = calculate_ten_day_payment(
            litres, fat, snf, month
        )
        print(f"{name}: rate Rs.{rate:.2f}, milk Rs.{milk:.2f}, "
              f"bonus Rs.{bonus:.2f}, total Rs.{total:.2f}")

if __name__ == "__main__":
    run_tests()
```

Analysis / Discussion

- The example assumes litres is the farmer's total supply for ten days.
- The bonus condition is strictly greater than 500, so exactly 500 litres does not qualify.
- The seasonal selection and bonus are separated into functions so later rule changes can be made with less duplicated code.

Output Screenshot:



Result

Thus, the calculator was adapted to the changed analyser precision, seasonal rates, and ten-day bonus requirements, and the revised logic was tested.